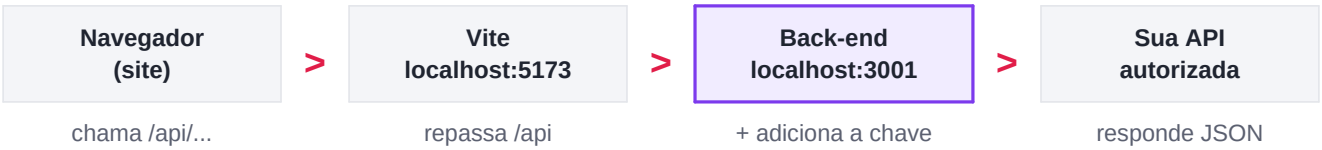
ANTONIO STREAMING PROJECT V1.6

Tutorial: conectando a sua API autorizada

Onde colocar a chave, como a API deve responder e o que esperar depois de ligá-la ao projeto.

Visão geral

O site **nunca** fala diretamente com a sua API. Ele chama o back-end do projeto (`server/index.js`), que lê a chave de um arquivo que fica só no seu computador, anexa a chave à requisição e repassa a resposta para o site. Assim, a chave nunca aparece no navegador.



1. Onde fica cada arquivo

Existem **dois** arquivos de configuração com papéis diferentes. A chave vai em apenas um deles.

```
catalogo_projeto/
|-- server/
|   |-- chave_api.env           <- A CHAVE ENTRA AQUI (API_KEY=)
|   |-- chave_api.env.example      modelo vazio (é o que vai para o GitHub)
|   |-- app.js                     back-end: rotas e caminhos da API
|   `-- catalog.js                 proxy que anexa a chave
|-- src/
|   `-- services/api.js            chamadas do site ao back-end
|-- .env.example
`-- .env                       <- dados PÚBLICOS do Supabase; NÃO é para a chave
```

Caminho do arquivo da chave, a partir da pasta do projeto:

```
catalogo_projeto\server\chave_api.env
```

Arquivo	O que vai nele	Visível no navegador?
`server/chave_api.env`	URL da API (API_BASE_URL) e **a chave (API_KEY)**	Não
`.env (raiz do projeto)`	Só os dados públicos do Supabase (login). Nada da API.	Sim

Atenção

Tudo que começa com `VITE_` é embutido no código que o navegador baixa. Por isso a chave **jamais** deve ser colocada no `.env` da raiz.

2. Passo a passo

1 Abra (ou crie) o arquivo da chave

No seu computador o arquivo já existe. Dentro da pasta do projeto, abra no Bloco de Notas:

```
notepad server\chave_api.env
```

Se você clonou o projeto do GitHub em outra máquina, o arquivo não vem junto. Crie a partir do modelo, na pasta do projeto:

```
Copy-Item server\chave_api.env.example server\chave_api.env
```

2 Preencha a URL e a chave

Cole os valores logo depois do =, **sem aspas e sem espaços**:

```
API_BASE_URL=https://api.seuprovedor.com/v1

# >>> A CHAVE ENTRA AQUI <<<
API_KEY=sk_live_SUA_CHAVE_AQUI
```

Variável	Para que serve	Exemplo
`API_BASE_URL`	Endereço base da API, **sem barra no final**.	`https://api.ex.com/v1`
`API_KEY`	Sua chave secreta.	`abc123...`
`API_KEY_HEADER`	Nome do cabeçalho que leva a chave.	`Authorization`
`API_KEY_PREFIX`	Texto antes da chave no cabeçalho (pode ficar vazio).	`Bearer`
`API_KEY_QUERY_PARAM`	Se preenchido, a chave vai na URL e o cabeçalho é ignorado.	`api_key`
`API_ALLOWED_PATHS`	Únicos caminhos da API que o site pode chamar. Protege a chave.	`/titles`
`API_CACHE_SECONDS`	Tempo que as respostas ficam em cache (economiza cota).	`60`
`PORT`	Porta do back-end local. Não altere sem necessidade.	`3001`

3 Diga como a sua API recebe a chave

Consulte a documentação do seu provedor e escolha a linha correspondente:

A documentação pede	Configure assim	O back-end envia

`Authorization: Bearer CHAVE`	`API_KEY_HEADER=Authorization`
	`API_KEY_PREFIX=Bearer`
	`API_KEY_QUERY_PARAM=`
		`Authorization: Bearer abc123`

`x-api-key: CHAVE`	`API_KEY_HEADER=x-api-key`
	`API_KEY_PREFIX=`
	`API_KEY_QUERY_PARAM=`
		`x-api-key: abc123`

`?api_key=CHAVE na URL`	`API_KEY_QUERY_PARAM=api_key`	`GET /titles?api_key=abc123`

4 Libere só os caminhos que o site usa

O back-end só repassa para a sua API os caminhos listados em `API_ALLOWED_PATHS`. Qualquer outro caminho é recusado **antes** de chegar à API, então ninguém consegue usar a sua chave em endpoints de conta ou cobrança do provedor. O padrão já cobre os endpoints da seção 3:

```
API_ALLOWED_PATHS=/titles
```

Se a sua API usar outros caminhos, liste todos separados por vírgula (ex.: `/movies,/series`) e ajuste as rotas em `server/app.js` (seção 3.5).

5 Reinicie o projeto

O back-end e o Vite só leem os arquivos de configuração quando iniciam. Pare o que estiver rodando (**Ctrl+C**) e execute:

```
npm run dev:all
```

Esse comando sobe o site (porta 5173) e o back-end (porta 3001) juntos. O login do Supabase precisa estar configurado (veja o README).

3. Modelo da API (sintaxe esperada)

O site faz chamadas **GET** e espera respostas em **JSON**. Ele nunca fala com a sua API: chama o back-end (`/api/catalog`, `/api/search`), que confere o login e então chama `API_BASE_URL` + o caminho abaixo, já com a chave.

3.1 Endpoints

Método	Caminho na sua API	Deve retornar	Usado em
`GET`	`/titles`	Array de títulos	Catálogo inteiro (o site chama /api/catalog)
`GET`	`/titles/search?q=texto`	Array de títulos	Busca do header (o site chama /api/search)
`GET`	`/titles/{id}`	Um objeto título	Reservado para uma futura página de detalhes

Exemplo da requisição que o back-end faz por você (com a configuração Bearer):

```
GET https://api.seuprovedor.com/v1/titles
Accept: application/json
Authorization: Bearer abc123
```

3.2 Formato de cada título

`/titles` e `/titles/search` devem devolver um **array** (começando com `[`) de objetos neste formato:

```
[
  {
    "id": 1,
    "title": "Horizonte de Vidro",
    "type": "filme",
    "year": 2026,
    "duration": "2h 14min",
    "ageRating": "14",
    "score": 9.1,
    "match": 98,
    "genres": ["Ficção Científica", "Drama"],
    "synopsis": "Quando uma cientista descobre que o céu...",
    "cast": ["Helena Prado", "Caio Monteiro"],
    "director": "Marina Albuquerque",
    "poster": "https://cdn.seuprovedor.com/posters/1.jpg",
    "backdrop": "https://cdn.seuprovedor.com/backdrops/1.jpg",
    "featured": true,
    "isNew": true,
    "progress": null
  }
]
```

3.3 Campos

Campo	Tipo	Obrig.	Regra / descrição
`id`	número ou texto	Sim	Único para cada título.
`title`	texto	Sim	Nome exibido.
`type`	texto	Sim	Exatamente `"filme"` ou `"serie"` (sem acento). Define as abas Filmes e Séries.
`genres`	array de textos	Sim	Gera as fileiras e os filtros por gênero. Use [] se não houver.
`cast`	array de textos	Sim	Elenco, mostrado nos detalhes. Use [] se não houver.
`score`	**número**	Sim	Nota de 0 a 10. Precisa ser número (9.1), não texto ("9.1").
`match`	número	Sim	Relevância de 0 a 100 (exibida como 98% relevante).
`year`	número	Sim	Ano de lançamento.
`duration`	texto	Sim	Livre: "2h 14min", "3 temporadas"...
`ageRating`	texto	Sim	Um de: "L", "10", "12", "14", "16", "18" (define a cor do selo).
`synopsis`	texto	Sim	Sinopse.
`director`	texto	Sim	Direção (filmes) ou criação (séries).
`poster`	URL ou null	Não	Imagem vertical (Top 10). Sem ela, o site gera uma arte.
`backdrop`	URL ou null	Não	Imagem horizontal (cards, banner, detalhes).
`featured`	true/false	Não	Coloca o título no banner rotativo do topo.
`isNew`	true/false	Não	Mostra o selo "Novo" e entra em Lançamentos.
`progress`	número 0 a 1 ou null	Não	Progresso assistido (0.42 = 42%). Entra em Continue assistindo.

3.4 Regras de resposta

- Status **200** e cabeçalho `Content-Type: application/json`.
- As listas devem vir como **array direto**. Se a sua API embrulhar a lista, por exemplo `{"results": [...]}`, é preciso adaptar o código (seção 3.5).
- Os nomes dos campos são sensíveis a maiúsculas: `ageRating` é diferente de `agerating`.
- Erros de autenticação devem usar 401 ou 403. O back-end repassa o mesmo status para o site.
- O back-end espera no máximo **10 segundos** por resposta; depois disso devolve 502.
- Respostas acima de **5 MB**, que não sejam JSON ou que redirecionem para outro endereço são recusadas (502).

3.5 Se a sua API tiver outro formato

Faça a conversão em `src/services/api.js`, sem mexer no resto do site. Exemplo para uma API que devolve `{"results": [...]}` com nomes em inglês:

```
// src/services/api.js
const toTitle = (raw) => ({
  id: raw.id,
  title: raw.name,
  type: raw.kind === 'series' ? 'serie' : 'filme',
  year: Number(raw.release_year),
  duration: raw.runtime_label,
  ageRating: String(raw.rating ?? 'L'),
  score: Number(raw.vote_average ?? 0),
  match: Math.round((raw.vote_average ?? 0) * 10),
  genres: raw.genres ?? [],
  synopsis: raw.overview ?? '',
  cast: raw.cast ?? [],
  director: raw.director ?? '',
  poster: raw.poster_url ?? null,
  backdrop: raw.backdrop_url ?? null,
});

export const getCatalog = () =>
  request('/catalog').then((data) => data.results.map(toTitle));

export const searchTitles = (query) =>
  request(`/search?q=${encodeURIComponent(query)}`)
    .then((data) => data.results.map(toTitle));
```

Se os caminhos da sua API forem outros (por exemplo `/movies` em vez de `/titles`), altere em `server/app.js` o texto passado para `proxyToApi(...)` nas rotas `/api/catalog` e `/api/search`, e inclua o novo caminho em `API_ALLOWED_PATHS`.

4. Resultados esperados

Depois dos 5 passos, confira cada item abaixo, nesta ordem.

4.1 No terminal

Ao rodar `npm run dev:all`, as linhas marcadas como **[api]** devem mostrar:

```
[api] [server] Back-end rodando em http://localhost:3001
[api] [server] Login (Supabase) configurado.
[api] [server] Chave da API carregada. Repassando -> https://api.seuprovedor.com/v1
```

Se aparecer `API não configurada: usando o catálogo de demonstração`, o `API_BASE_URL` ou o `API_KEY` está vazio.

4.2 No navegador: status do back-end

Abra `http://localhost:5173/api/health`. O esperado é:

```
{"ok":true,"login":true,"api":true}
```

Este endereço nunca mostra chaves, só se elas foram carregadas.

4.3 No navegador: dados da sua API

Entre no site com um usuário liberado. Abra o **F12 > Rede** e clique na chamada `catalog`: a resposta é o JSON da sua API, no formato da seção 3.2. Sem login, os endereços `/api/...` respondem 401, de propósito.

4.4 No site

- O selo amarelo **"Modo demonstração · dados mockados"** no canto inferior direito **desaparece**.
- O banner, as fileiras, o Top 10 e os filtros passam a mostrar os títulos da **sua** API.
- Títulos com `poster`/`backdrop` exibem as imagens reais; os demais continuam com a arte gerada.
- A busca do header passa a consultar `/titles/search` na sua API.
- No navegador (F12 > Rede), as chamadas aparecem como `/api/...` e **sem** a chave.

> **Pronto!**
> Se os quatro testes acima passaram, a API está conectada e a chave está protegida no back-end.

4.5 Problemas comuns

Quando algo falha, o site mostra *"Não foi possível carregar o catálogo"* com o código do erro. Use a tabela:

Você vê	Causa provável	Como resolver
Selo amarelo continua	API_BASE_URL ou API_KEY vazio, ou o projeto não foi reiniciado.	Preencha server/chave_api.env e rode npm run dev:all de novo.
Erro 503	Chave/URL da API ou do Supabase vazia em server/chave_api.env.	Preencha as variáveis indicadas na mensagem, salve e reinicie.
Erro 401 ou 403	Chave inválida, expirada ou enviada no formato errado.	Confira a chave e o passo 3 (cabeçalho, prefixo ou parâmetro).
Erro 404	Caminho fora de API_ALLOWED_PATHS, caminho diferente na sua API ou barra sobrando na URL base.	Ajuste API_ALLOWED_PATHS e as rotas em server/app.js (seção 3.5); revise API_BASE_URL.
Erro 502	API fora do ar, URL errada, resposta acima de 10 s ou 5 MB, ou que não é JSON.	Teste a URL da API diretamente e confira API_BASE_URL.
Página em branco ou erro no console	Formato do JSON diferente do esperado (ex.: lista embrulhada, score como texto).	Veja a resposta em F12 > Rede > catalog, compare com a seção 3.2 e adapte com a seção 3.5.
Erro 405	O site tentou um método diferente de GET.	O back-end só permite leitura. Peça a liberação de outros métodos se precisar.
Erro 429	Muitas requisições do mesmo IP em pouco tempo.	Aguarde um minuto. O limite protege a sua cota contra abuso.

5. Segurança

- `server/chave_api.env` está no `.gitignore` e não vai para o GitHub. Não renomeie o arquivo.
- Nunca cole a chave em `src/`, no `.env` da raiz, no README ou em mensagens.
- Antes de cada `git push`, confira com `git status` que o arquivo da chave não aparece.
- Se a chave for publicada por engano, **gere uma chave nova** no provedor. Apagar o arquivo não basta: ela continua no histórico do Git.
- Na Vercel, as chaves não vão em arquivo: cadastre as mesmas variáveis em Settings > Environment Variables (veja o README).